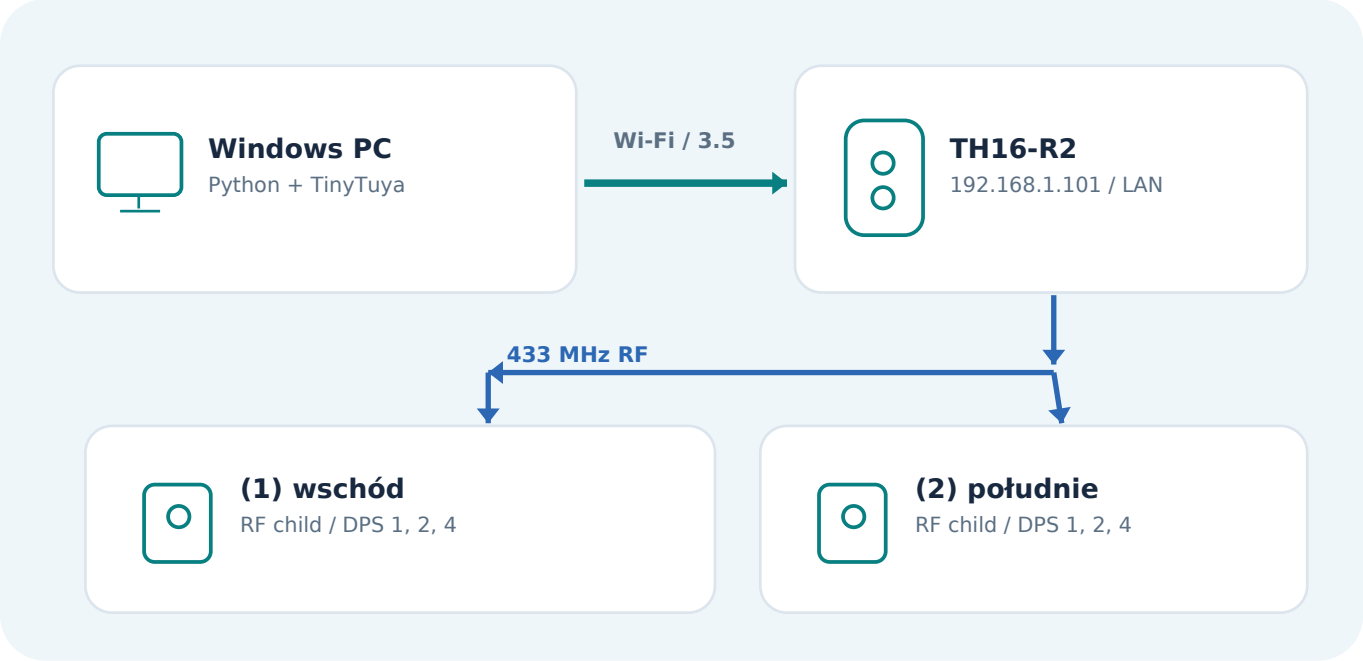
ILLUSTRATED LEARNING GUIDE

Understand your TH16-R2 Python weather reader

A software architecture guide for beginners, with a special focus on the **DPS mapping** dictionary and the **decode_readings()** function.



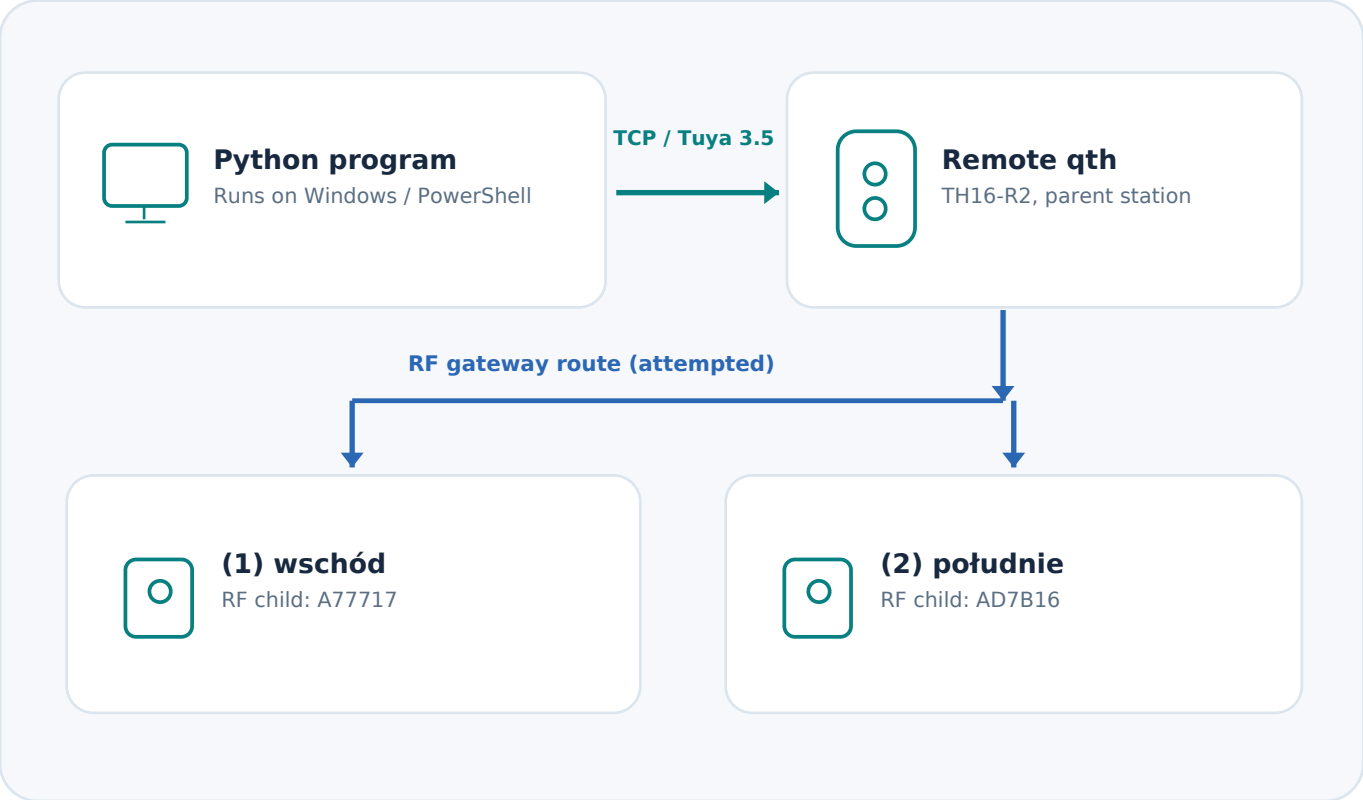
READ LOOK UP CONVERT DISPLAY

Learning path: First see the system; next see the mapping; then trace one real value from a sensor to the screen.

Based on your exact script: `th16_all_sensors(2).py`. The verified main-station example is raw DPS **1 = 102** and **2 = 66**, which displays as **10.2°C** and **66%**. RF local communication is attempted, not yet proven.

1. What the program talks to

Your program has a **parent Wi-Fi station** and two **RF child sensors**. The children do not need individual LAN IP addresses.



devices.json

Stores devices, parent/child links, local keys and DPS mapping.

tinytuya.json

Optional cloud credentials; used only when --cloud is requested.

Facts vs. assumptions

Confirmed: The main station responds at `192.168.1.101` using protocol **3.5**. DPS 1 and 2 match its temperature and humidity. **Not confirmed:** whether the gateway returns each RF child's measurements over LAN.

2. Functions: one job each

In Python, **main()** is the coordinator. Other functions specialize in configuration, communication, conversion and display.

1 load_devices()

Find TH16-R2 and RF children in devices.json.

↓

2 main() creates gateway

Build a TinyTuya Device with station ID, IP, key and 3.5.

↓

3 read_main_station()

Call gateway.status(), then extract numbered DPS.

↓

4 read_rf_sensor()

Try each child via parent=gateway and cid=node_id.

↓

5 show_readings()

Call decode_readings(); print temperature/humidity.

↓

Important: `read_main_station()` and `read_rf_sensor()` both eventually use `show_readings()`. That function calls `decode_readings()`. The same mapping logic therefore works for all configured sensors.

3. What does mapping mean?

Think of the mapping as a **translation dictionary**. Raw DPS says what number arrived; mapping says what that number means.

A. LIVE DPS

```
"1": 102
"2": 66
```

Values returned now by your weather station.

B. MAPPING

```
"1" → temperature
scale = 1
"2" → humidity
scale = 0
```

Definitions from devices.json.

C. READINGS

10.2°C
66%

Human-readable decoded output.

Actual mapping structure (shortened for learning)

```
"mapping": {
  "1": {"code": "va_temperature",
        "values": {"scale": 1}},
  "2": {"code": "va_humidity",
        "values": {"scale": 0}}
}
```

Read this like a set of instructions:

"1" is a DPS key, not a temperature. **"va_temperature"** is its meaning. **"scale": 1** means divide by 10. **"scale": 0** means divide by 1 (no numeric change).

Not live data: mapping normally stays the same while the measured values change every time you ask the device for status.

4. Follow one real temperature

This follows the **successful path** through `decode_readings()` for TH16-R2 DPS **"1" = 102**. Python chooses the DPS key using a loop; the source code does **not** assign `dp_number = "1"` manually.

01

The loop picks a key

for dp_number in mapping:

In this mapping, the first key is "1". The loop sets it automatically.

02

Find that key's definition

dp_info = mapping[dp_number]

The entry for "1" describes temperature and contains its scale.

03

Get the measurement name

measurement = dp_info.get("code")

Here, measurement becomes "va_temperature".

04

Find the live measurement

raw_value = raw_dps[dp_number]

After checking the key exists, retrieve the raw value 102.

05

Read conversion instructions

scale = int(values_info.get("scale", 0))

First get values_info from dp_info; the temperature scale is 1.

06

Calculate and save

readings[measurement] = converted_value

102 / (10 ** 1) = 10.2; save it as "va_temperature".

Important checks in the real function

It skips unsupported measurement names, missing DPS keys and values that are not numeric. This diagram shows only a **successful** iteration.

Next iteration: humidity

The loop picks key "2". Raw value = 66; scale = 0. `66 / (10 ** 0) = 66`. Both results are kept in `readings`.

5. Decode: read each line slowly

A shortened excerpt of the function from your file. The real code also checks supported measurement names, missing DPS keys and numeric types (see page 5).

```
def decode_readings(raw_dps, device_info):
    readings = {}
    mapping = device_info.get("mapping", {})
    for dp_number in mapping:
        dp_info = mapping[dp_number]
        measurement = dp_info.get("code")
        raw_value = raw_dps[dp_number]  # after checks
        values_info = dp_info.get("values", {})
        scale = int(values_info.get("scale", 0))
        converted = raw_value / (10 ** scale)
        readings[measurement] = converted
    return readings
```

readings = {}
Make an empty dictionary that will hold answers.

mapping = device_info.get("mapping", {})
Fetch the translation rules; use {} if absent.

for dp_number in mapping:
Repeat the next block once for each DPS key.

dp_info = mapping[dp_number]
Get the definition of this one DPS key.

values_info = dp_info.get("values", {})
Take the nested settings dictionary.

scale = int(values_info.get("scale", 0))
Find the scale; use 0 if missing; convert to integer.

readings[measurement] = converted
Store the result under a meaningful name.

Difference between [] and .get(): `dictionary["x"]` expects key x to exist; `dictionary.get("x", 0)` returns 0 if it does not. The latter avoids a missing-key error.

6. How functions pass the data

The program uses small functions as a relay. Each receives a value, does a job and returns a new value.



Example call: `show_readings(device_name=main_station["name"], raw_dps=raw_dps, device_info=main_station, source="LAN")`. The **device_info** parameter contains the mapping.

7. RF sensors and the cloud option

The main station is confirmed to respond locally. Each RF sensor is **attempted** through the parent connection; the code does not guarantee that RF child data will be available.



Why the cloud response needs another mapping step

Local form: `{"1": 102, "2": 66}`

Cloud-style form: `[{"code":"va_temperature","value":102}, ...]`

`get_cloud_dps()` uses mapping to turn the cloud measurement name back into a numbered DPS. Then `decode_readings()` can process local and cloud data in the same way.

Reminder: RF LAN routing and cloud availability are hardware/account dependent; missing data is reported, not invented.

8. Run it and read the output

Keep the Python file and devices.json in the same folder. Your script looks for JSON files next to the Python file by default.

```
# Install once (Windows PowerShell)
py -m pip install tinytuya

# Run local reading
py "th16_all_sensors(2).py"

# Optional: try cloud for missing RF readings
py "th16_all_sensors(2).py" --cloud

# Display supported options
py "th16_all_sensors(2).py" --help
```

Example output from the main station

```
Device: Remote qth
Source: LAN
Temperature: 10.2 °C
Humidity: 66 %
Raw DPS: {"1": 102, "2": 66, "23": -8, "24": 0}
```

What to check if a reading is missing

- **No TH16-R2 found:** check the parent model and sub fields inside devices.json.
- **No main-station DPS:** verify the station IP, protocol 3.5, Device ID and Local Key.
- **RF sensor has no DPS:** the parent may not expose that child locally; try --cloud if configured.
- **A DPS key is absent:** decode_readings() skips it rather than inventing a value.

Security: Never publish devices.json or tinytuya.json. They can contain working device Local Keys or cloud credentials. This guide intentionally contains no secret values.

9. Reference card & practice

You can use this page as a compact reminder when you reopen the script.

Key idea: raw_dps supplies readings; mapping supplies the rules for interpreting them.

DEVICE	DPS "1"	DPS "2"	DPS "4"
TH16-R2 station	Temp /10	Humidity	-
(1) wschód	Temp /10	Humidity	Battery
(2) południe	Temp /10	Humidity	Battery

Tiny exercise - can you decode this?

Suppose the weather station sends `raw_dps = {"1": 187, "2": 54}` and your mapping is unchanged.

Temperature

DPS 1 = 187
scale = 1
187 ÷ 10 = 18.7°C

Humidity

DPS 2 = 54
scale = 0
54 ÷ 1 = 54%

Four words to remember

DPS	Raw numbered value received from a Tuya device.
mapping	Dictionary of DPS meaning, type, unit and scale.
decode_readings()	Looks up mapping and transforms raw values.
show_readings()	Prints those decoded values for a person.